

Nama : Muhammad Ridwan Al Faqih

Kelas : XI RPL 1

PROJEK SNAKE GAME ULAR DENGAN C++ / PBTGM

- A. SOURCE CODE** Berikut adalah implementasi kode program Snake Game menggunakan bahasa C++ yang telah dimodifikasi dengan fitur *Input Nama Player*, *Penyimpanan High Score*, dan perbaikan rendering agar tidak berkedip (*flicker-free*).

```
#include <iostream>
#include <vector>
#include <conio.h>
#include <windows.h>
#include <string>

using namespace std;

// --- KONFIGURASI & STRUKTUR DATA ---
const int lebar = 20;
const int tinggi = 15;

struct Poin { int x, y; };

// Perbaikan Bug: Menambahkan keyword 'enum'
enum Arah { DIAM = 0, KIRI, KANAN, ATAS, BAWAH };
Arah dir;

// Variabel Global
string namaPlayer;
Poin kepala, buah;
vector<Poin> ekor;
bool gameOver;
int skor;
int delayTime = 250; // Kecepatan awal

// --- SISTEM UTILITAS (WINDOWS API) ---

// Fungsi: Mematikan kursor kedip agar visual map bersih
void SembunyikanKursor() {
    HANDLE out = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO cursorInfo;
    GetConsoleCursorInfo(out, &cursorInfo);
    cursorInfo.bVisible = false;
    SetConsoleCursorInfo(out, &cursorInfo);
}
```

```

// Fungsi: Memindahkan kursor ke 0,0 tanpa menghapus layar
(Anti-Flicker)
void ResetKursor() {
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE),
    {0,0});
}

// --- LOGIKA GAME ---

// Penjelasan: Menu awal untuk input nama
void TampilkanMenu() {
    system("cls");
    cout << "===== " << endl;
    cout << "          SNAKE GAME ENGINE          " << endl;
    cout << "===== " << endl;
    cout << " MASUKKAN NAMA ANDA: ";
    getline(cin, namaPlayer);
    if(namaPlayer == "") namaPlayer = "User";

    cout << "\n Halo, " << namaPlayer << "!" << endl;
    cout << " Kontrol: W, A, S, D | X: Keluar" << endl;
    cout << " Tekan tombol apa saja untuk MULAI..." << endl;
    cout << "===== " << endl;
    _getch();
    system("cls");
}

// Penjelasan: Reset variabel ke kondisi awal
void Setup() {
    SembunyikanKursor();
    gameOver = false;
    dir = DIAM;
    kepala = {lebar / 2, tinggi / 2};
    buah = {rand() % lebar, rand() % tinggi};
    skor = 0;
    ekor.clear();
    delayTime = 250;
}

// Penjelasan: Logika Kecepatan 4 Tahap sesuai permintaan
void UpdateKecepatan() {
    if (skor >= 10 && skor <= 30)        delayTime = 250; //
    Super Lambat
    else if (skor >= 40 && skor <= 80)    delayTime = 150; //
    Lambat

```

```

        else if (skor >= 90 && skor <= 200) delayTime = 80; //
Cepat
        else if (skor >= 210)                delayTime = 40; //
Super Cepat
    }

// Penjelasan: Menggambar Map (Skala 1:1 dengan double
character)
void Render() {
    ResetKursor();
    string map = "";

    for (int i = 0; i < lebar + 2; i++) map += "##";
    map += "\n";

    for (int i = 0; i < tinggi; i++) {
        for (int j = 0; j < lebar; j++) {
            if (j == 0) map += "##";

            if (i == kepala.y && j == kepala.x) map += "[";
// Kepala
            else if (i == buah.y && j == buah.x) map += " @";
// Buah
            else {
                bool isEkor = false;
                for (auto& e : ekor) {
                    if (e.x == j && e.y == i) { map += "(";
isEkor = true; break; }
                }
                if (!isEkor) map += " ";
            }
            if (j == lebar - 1) map += "##";
        }
        map += "\n";
    }

    for (int i = 0; i < lebar + 2; i++) map += "##";
    map += "\n PLAYER: " + namaPlayer;
    map += " | SKOR: " + to_string(skor);
    map += " | STATUS: " + (delayTime < 100 ? (string)"TURBO"
: (string)"NORMAL") + "\n";

    cout << map;
}

void Input() {
    if (_kbhit()) {

```

```

        switch (_getch()) {
            case 'a': if(dir != KANAN) dir = KIRI; break;
            case 'd': if(dir != KIRI) dir = KANAN; break;
            case 'w': if(dir != BAWAH) dir = ATAS; break;
            case 's': if(dir != ATAS) dir = BAWAH; break;
            case 'x': gameOver = true; break;
        }
    }
}

void Logika() {
    if (dir == DIAM) return;

    Poin posisiLama = kepala;

    switch (dir) {
        case KIRI: kepala.x--; break;
        case KANAN: kepala.x++; break;
        case ATAS: kepala.y--; break;
        case BAWAH: kepala.y++; break;
    }

    // Tabrak Tembok
    if (kepala.x >= lebar || kepala.x < 0 || kepala.y >=
tinggi || kepala.y < 0)
        gameOver = true;

    // Tabrak Ekor
    for (auto& e : ekor) if (e.x == kepala.x && e.y ==
kepala.y) gameOver = true;

    // Makan Buah
    if (kepala.x == buah.x && kepala.y == buah.y) {
        skor += 10;
        UpdateKecepatan(); // Update transmisi gigi
        buah = {rand() % lebar, rand() % tinggi};
        ekor.insert(ekor.begin(), posisiLama);
    } else if (!ekor.empty()) {
        ekor.insert(ekor.begin(), posisiLama);
        ekor.pop_back();
    }
}

int main() {
    TampilkanMenu();
    Setup();
}

```

```

while (!gameOver) {
    Render();
    Input();
    Logika();
    Sleep(delayTime);
}

system("cls");
cout << "===== " << endl;
cout << "          GAME OVER!          " << endl;
cout << " PLAYER      : " << namaPlayer << endl;
cout << " SKOR AKHIR: " << skor << endl;
cout << "===== " << endl;
cout << " Tekan tombol apapun untuk keluar." << endl;
_getch();

return 0;
}

```

B. PENJELASAN SINTAKS DAN FUNGSI

Berikut adalah draf materi untuk bagian A. **PENJELASAN SINTAKS DAN FUNGSI** yang dirancang khusus agar terlihat profesional untuk laporan tugas, namun tetap memiliki sentuhan bahasa teknis anak muda yang *up-to-date*.

A. PENJELASAN SINTAKS DAN FUNGSI

Bagian ini menguraikan komponen teknis dari *Snake Game Engine* yang dikembangkan menggunakan bahasa pemrograman C++. Penjelasan dibagi menjadi tiga kategori utama: *Library*, Struktur Data, dan Fungsi Inti.

1. Library & Header Files

Program ini memanfaatkan beberapa *library* standar C++ dan Windows API untuk menangani visual serta *input* secara *real-time*.

Library	Fungsi & Kegunaan
<iostream>	Menangani alur input/output standar (seperti cout untuk mencetak map).
<vector>	Digunakan untuk menyimpan koordinat ekor ular secara dinamis (fleksibel).
<conio.h>	Mengaktifkan fungsi _kbhit() dan _getch() untuk <i>input</i> tanpa menekan Enter.
<windows.h>	Library vital untuk memanipulasi konsol (kecepatan Sleep dan posisi kursor).
<string>	Menangani tipe data teks, terutama untuk penyimpanan nama player.

2. Struktur Data & Variabel Global

Logika permainan dibangun di atas struktur data yang merepresentasikan koordinat dan arah.

- struct Poin { int x, y; }

Struktur ini adalah fondasi koordinat Kartesius dalam game. Setiap bagian tubuh ular dan posisi buah direpresentasikan sebagai titik x dan y.

- enum Arah { DIAM, KIRI, KANAN, ATAS, BAWAH }

Enumerasi digunakan untuk menciptakan tipe data khusus yang membatasi arah gerak ular. Ini mencegah penggunaan angka "ajaib" (*magic numbers*) dan membuat kode lebih mudah dibaca (*human-readable*).

- vector<Poin> ekor

Berperan sebagai *container* dinamis. Setiap kali ular makan buah, koordinat baru disisipkan ke dalam *vector* ini sehingga ular terlihat memanjang.

3. Fungsi Utama (Core Functions)

Nama Fungsi	Penjelasan Teknis
SembunyikanKursor()	Mengakses <i>Console Cursor Info</i> untuk mematikan visibilitas kursor (<code>_</code>). Hal ini dilakukan agar layar tetap bersih saat proses <i>rendering</i> berjalan.
ResetKursor()	Mengembalikan kursor ke koordinat (0,0) di pojok kiri atas. Teknik ini jauh lebih efisien dan stabil dibandingkan <code>system("cls")</code> yang sering menyebabkan layar berkedip (<i>flickering</i>).
TampilkanMenu()	Berfungsi sebagai fase inisialisasi. Di sini terdapat proses <i>String Input</i> untuk menyimpan nama player menggunakan <code>getline</code> .
UpdateKecepatan()	Logika Utama Kecepatan. Fungsi ini melakukan pengecekan terhadap variabel skor. Jika skor mencapai <i>threshold</i> tertentu (10, 40, 90, 210), maka nilai variabel <code>delayTime</code> akan diperbarui untuk meningkatkan kesulitan.
Render()	<i>Graphic Engine</i> sederhana. Menggunakan teknik <i>Double-Character</i> (setiap 1 unit visual diwakili 2 karakter) untuk menjaga aspek rasio 1:1 agar map tidak terlihat gepeng di terminal.
Input()	Menggunakan mekanisme <i>Non-blocking Input</i> . Fungsi ini mengecek apakah ada interaksi pada keyboard tanpa menghentikan aliran logika permainan.

Nama Fungsi	Penjelasan Teknis
Logika()	Otak dari permainan. Menangani deteksi tabrakan (<i>Collision Detection</i>) baik terhadap tembok maupun tubuh sendiri, serta mengatur algoritma perpindahan koordinat dari kepala ke ekor.

4. Algoritma Kecepatan Adaptif

Sistem ini menggunakan algoritma berbasis kondisi (if-else) untuk mengatur tingkat kesulitan secara otomatis:

1. Stage 1 (Super Lambat): delayTime = 250ms (Skor 0-30).
2. Stage 2 (Lambat): delayTime = 150ms (Skor 40-80).
3. Stage 3 (Cepat): delayTime = 80ms (Skor 90-200).
4. Stage 4 (Turbo/Extreme): delayTime = 40ms (Skor 210+).

Analogi: Sistem ini bekerja layaknya transmisi otomatis pada kendaraan, di mana beban kerja (skor) akan menentukan seberapa cepat mesin (looping program) berputar.

C. TAMPILAN OUTPUT

```
E:\Documents\Joki Tugas\M f x + v
=====
SNAKE GAME ENGINE
=====
MASUKKAN NAMA ANDA: Ridwan

Halo, Ridwan!
Kontrol: W, A, S, D | X: Keluar
Tekan tombol apa saja untuk MULAI...
=====
|
```

```
E:\Documents\Joki Tugas\M f x + v
#####
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##      OOOOOO[]                      ##
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##                                     ##
##      @                             ##
##                                     ##
##                                     ##
#####
PLAYER: Ridwan | SKOR: 60 | STATUS: NORMAL
```

```
E:\Documents\Joki Tugas\M f x + v
=====
                GAME OVER!
PLAYER       : Ridwan
SKOR AKHIR: 60
=====
Tekan tombol apapun untuk keluar.
```

Demikian hasil kerja saya sekian terimakasih.